COSC2637 - Big Data Processing

Lecture 10

Spark Streaming - Scala

RMIT UNIVERSITY

Part I
Scala Basics - 1

What is Scala?

- Scala is a modern multi-paradigm programming language designed to express common programming patterns in a concise, elegant, and type-safe way.
  - Scala smoothly integrates the features of object-oriented and functional languages.
- Scala has been created by Martin Odersky and he released the first version in 2003.

Online source: https://www.tutorialspoint.com/scala/scala_classes_objects.htm

What is Scala?

- Scala is a modern multi-paradigm programming language designed to express common programming patterns in a concise, elegant, and type-safe way.
  - Scala smoothly integrates the features of object-oriented and functional languages.
- Scala has been created by Martin Odersky and he released the first version in 2003.

Online source: https://www.tutorialspoint.com/scala/scala_classes_objects.htm

Spark Programming – Why Scala?

- The most difficult thing for big data developers today is choosing a programming language for big data applications.
  - Python and R programming, are the languages of choice among data scientists for building machine learning models
  - Java remains the go-to programming language for developing Hadoop applications.

Spark Programming – Why Scala?

- The most difficult thing for big data developers today is choosing a programming language for big data applications.
  - Python and R programming, are the languages of choice among data scientists for building machine learning models
  - Java remains the go-to programming language for developing Hadoop applications.

RMIT Classification: Trusted

Spark Programming – Why Scala?

- The most difficult thing for big data developers today is choosing a programming language for big data applications.
  - Python and R programming, are the languages of choice among data scientists for building machine learning models
  - Java remains the go-to programming language for developing Hadoop applications.

Spark Programming – Why Scala?

- With the advent of various big data frameworks like Apache Spark- Scala programming language has gained prominence amid big data developers. Why?

Spark Programming – Why Scala?

- With the advent of various big data frameworks like Apache Spark- Scala programming language has gained prominence amid big data developers. Why?



Spark Programming – Why Scala?

- With the advent of various big data frameworks like Apache Spark- Scala programming language has gained prominence amid big data developers. Why?



- 88% of Spark users were using Scala for Apache Spark, 22% were using Python and 44% were using Java - making it the de facto language for Spark programming.

Object-oriented and Functional languages

- **Scala is object-oriented**
  - Scala is a pure object-oriented language in the sense that every value is an object. Types and behaviour of objects are described by classes
  - Classes are extended by subclassing and a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

Object-oriented and Functional languages

- **Scala is object-oriented**
  - Scala is a pure object-oriented language in the sense that every value is an object. Types and behaviour of objects are described by classes
  - Classes are extended by subclassing and a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

Object-oriented and Functional languages

- **Scala is object-oriented**
  - Scala is a pure object-oriented language in the sense that every value is an object. Types and behaviour of objects are described by classes
  - Classes are extended by subclassing and a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

Object-oriented and Functional languages

- **Scala is object-oriented**
  - Scala is a pure object-oriented language in the sense that every value is an object. Types and behaviour of objects are described by classes
  - Classes are extended by subclassing and a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

Object-oriented and Functional languages

- **Scala is object-oriented**
  - Scala is a pure object-oriented language in the sense that every value is an object. Types and behaviour of objects are described by classes
  - Classes are extended by subclassing and a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

In object-oriented programming languages, a mixin (or mix-in) is a class that contains methods for use by other classes without having to be the parent class of those other classes.

RMIT Classification: Trusted

Object-oriented and Functional languages

- **Scala is object-oriented**
  - Scala is a pure object-oriented language in the sense that every value is an object. Types and behaviour of objects are described by classes
  - Classes are extended by subclassing and a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

In object-oriented programming languages, a mixin (or mix-in) is a class that contains methods for use by other classes without having to be the parent class of those other classes.

- How other classes gain access to the mixin's methods depends on the language.
- Mixins are sometimes described as being "included" rather than "inherited".

RMIT Classification: Trusted

Object-oriented and Functional languages

- **Scala is object-oriented**
  - Scala is a pure object-oriented language in the sense that every value is an object. Types and behaviour of objects are described by classes
  - Classes are extended by subclassing and a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

In Scala, Traits helps implement mixin behaviour.

In object-oriented programming languages, a mixin (or mix-in) is a class that contains methods for use by other classes without having to be the parent class of those other classes.

- How other classes gain access to the mixin's methods depends on the language.
- Mixins are sometimes described as being "included" rather than "inherited".

Object-oriented and Functional languages

- **Scala is functional**
  - Scala is also a functional language in the sense that
    - every function is a value and
    - every value is an object

Object-oriented and Functional languages

- **Scala is functional**
  - Scala is also a functional language in the sense that
    - every function is a value and
    - every value is an object

Object-oriented and Functional languages

- **Scala is functional**
  - Scala is also a functional language in the sense that
    - every function is a value and
    - every value is an object

    so ultimately every function is an object.

Scala Variable Declaration

- The type of a variable is specified

```
val VariableName : DataType = [Initial Value]
```

```
var VariableName : DataType = [Initial Value]
```

For example: val myVar : String = “Foo”;

Scala Variable Declaration

- The type of a variable is specified

```
val VariableName : DataType = [Initial Value]
```

immutable variable

```
var VariableName : DataType = [Initial Value]
```

mutable variable

For example: val myVar : String = “Foo”;

Scala Variable Declaration

- The type of a variable is specified

```
val VariableName : DataType = [Initial Value]
```

immutable variable

```
var VariableName : DataType = [Initial Value]
```

mutable variable

For example: val myVar : String = “Foo”;

- When you assign an initial value to a variable, the Scala compiler can figure out the type of the variable based on the value assigned to it. This is called **variable type inference**.

  For example: val myVar = “Foo”;
  val myVal = 10;

Scala Variable Declaration

- The type of a variable is specified

```
val VariableName : DataType = [Initial Value]
```

immutable variable

```
var VariableName : DataType = [Initial Value]
```

mutable variable

For example: var myVar : String = “Foo”;

- When you assign an initial value to a variable, the Scala compiler can figure out the type of the variable based on the value assigned to it. This is called variable type inference.

  For example: var myVar = “Foo”;
  var myVal = 10;

- Scala supports multiple assignments

```
val (myVar1: Int, myVar2: String) = (40, "Foo")
```

Scala Variable Declaration

- The type of a variable is specified

```
val VariableName : DataType = [Initial Value]
```

immutable variable

```
var VariableName : DataType = [Initial Value]
```

mutable variable

For example: var myVar : String = “Foo”;

- When you assign an initial value to a variable, the Scala compiler can figure out the type of the variable based on the value assigned to it. This is called variable type inference.

  For example: var myVar = “Foo”;
  var myVal = 10;

- Scala supports multiple assignments

```
val (myVar1: Int, myVar2: String) = (40, "Foo")
```

or

```
val (myVar1, myVar2) = (40, "Foo")
```

Scala Function Definitions

- A Scala function definition has the following form

```
def functionName ([list of parameters]):[return type]={
      function body
      return [expr]
}
```

Scala Function Definitions

- A Scala function definition has the following form

```
def functionName ([list of parameters]):[return type]={
      function body
      return [expr]
}
```

Following is the function which will add two integers and return their sum

```
object add {
    def addInt( a:Int, b:Int ) : Int = {
         var sum:Int = 0
         sum = a + b
         return sum
    }
}
```

Scala Function Definitions

- A Scala function definition has the following form

```
def functionName ([list of parameters]):[return type]={
      function body
      return [expr]
}
```

Following is the function which will add two integers and return their sum

```
object add {
    def addInt( a:Int, b:Int ) : Int = {
         var sum:Int = 0
         sum = a + b
         return sum
    }
}
```

A function, that does not return anything can return a **Unit** that is equivalent to **void** in Java and indicates that function does not return anything.

Scala Function Definitions

- The functions which do not return anything in Scala, they are called procedures

```
object Hello{
   def printMe( ) : Unit = {
      println("Hello, Scala!")
   }
}
```

Scala Calling Functions

- The standard way to call a function

```
functionName( list of parameters )
```

- A function is being called using an instance of the object method

```
[instance.]functionName( list of parameters )
```

Scala Calling Functions

- An example

```
object Demo {
        def main(args: Array[String]) : Unit = {
                println( "Returned Value : " + addInt(5,7) );
        }

        def addInt( a:Int, b:Int ) : Int = {
                var sum:Int = 0
                sum = a + b
                return sum
        }
}
```

Demo.scala

Scala - Anonymous Function (Lambda Function)

Scala - Anonymous Function (Lambda Function)

- The first syntax shows that expression after "=>" evaluates to a particular value whereas a list of the variable before "=>" used for evaluating the expression.

```
('first_variable':'data_type', 'second_variable':'data_type') => "certain_expression"
```

Scala - Anonymous Function (Lambda Function)

- The first syntax shows that expression after "=>" evaluates to a particular value whereas a list of the variable before "=>" used for evaluating the expression.

```
('first_variable':'data_type', 'second_variable':'data_type') => "certain_expression"
```

Example: `(var1:String:var2:String) => var1 + var2`

Scala - Anonymous Function (Lambda Function)

- The first syntax shows that expression after "=>" evaluates to a particular value whereas a list of the variable before "=>" used for evaluating the expression.

```
('first_variable':'data_type', 'second_variable':'data_type') => "certain_expression"
```

Example: (var1:String:var2:String) => var1 + var2

- The second syntax above works like a placeholder where it accepts the value as a 'wild card' character only once, and then the operator is operated among them.

(_:'data_type')operator(_:'data_type')

Scala - Anonymous Function (Lambda Function)

- The first syntax shows that expression after "=>" evaluates to a particular value whereas a list of the variable before "=>" used for evaluating the expression.

```
('first_variable':'data_type', 'second_variable':'data_type') => "certain_expression"
```

Example: `(var1:String:var2:String) => var1 + var2`

- The second syntax above works like a placeholder where it accepts the value as a 'wild card' character only once, and then the operator is operated among them.

```
(_:'data_type')operator(_:'data_type')
```

Example: `(_:String)+(_:String)`

Scala - Anonymous Function

An example of anonymous function

```
object anonymousDemo {
     def main(args: Array[String]) {
         var function1 = (var1:Int, var2:Int) => var1 + var2
         var function2 = (_:Int) + (_:Int)

        // function call
         println(function1(5, 5))
         println(function2(7, 3))
      }
}
```

Scala - Anonymous Function

An example of anonymous function

```
object anonymousDemo {
     def main(args: Array[String]) {
         var function1 = (var1:Int, var2:Int) => var1 + var2
         var function2 = (_:Int) + (_:Int)

         // function call
         println(function1(5, 5))
         println(function2(7, 3))
      }
}
```

End of Part I

Part II
Scala Basics – 2
and
Execute Scala Programs

Scala - Closures

- A **closure** is a function, whose return value depends on the value of one or more variables declared outside this function.

Scala - Closures

- A **closure** is a function, whose return value depends on the value of one or more variables declared outside this function.

The following piece of code with anonymous function.

```
val multiplier = (i:Int) => i * 10
```

The only variable used in the function body, i * 10 , is i, which is defined as a parameter to the function.

Scala - Closures

- A **closure** is a function, whose return value depends on the value of one or more variables declared outside this function.

The following piece of code with anonymous function.

```
val multiplier = (i:Int) => i * 10
```

The only variable used in the function body, i * 10 , is i, which is defined as a parameter to the function.

```
var factor = 3
val multiplier = (i:Int) => i * factor
```

There are two free variables in multiplier: **i** and **factor**.
The function references factor and reads its current value each time.

Scala - Closures

```
object Demo {
      def main(args: Array[String]) {
         println( "multiplier(1) value = " + multiplier(1) )
         println( "multiplier(2) value = " + multiplier(2) )
       }

       var factor = 3
       val multiplier = (i:Int) => i * factor
}
```

a closure

Demo1.scala

Scala - Traits

- A trait encapsulates method and field definitions, which can then be reused by mixing them into classes.
- Unlike class inheritance, in which each class must inherit from just one superclass, a class can mix in any number of traits.

Scala - Traits

- A trait encapsulates method and field definitions, which can then be reused by mixing them into classes.
- Unlike class inheritance, in which each class must inherit from just one superclass, a class can mix in any number of traits.

Scala - Traits

- A trait definition looks just like a class definition except that it uses the keyword **trait**.

Scala - Traits

- A trait definition looks just like a class definition except that it uses the keyword **trait**.

```
trait Equal {
      def isEqual(x: Any): Boolean
      def isNotEqual(x: Any): Boolean = !isEqual(x)
}
```

This trait consists of two methods **isEqual** and **isNotEqual**.

Scala - Traits

- A trait definition looks just like a class definition except that it uses the keyword **trait**.

```
trait Equal {
      def isEqual(x: Any): Boolean
      def isNotEqual(x: Any): Boolean = !isEqual(x)
}
```

This trait consists of two methods **isEqual** and **isNotEqual**.

Scala - Traits

- A trait definition looks just like a class definition except that it uses the keyword **trait**.

```
trait Equal {
      def isEqual(x: Any): Boolean
      def isNotEqual(x: Any): Boolean = !isEqual(x)
}
```

function

function

This trait consists of two methods **isEqual** and **isNotEqual**.

Scala - Traits

- A trait definition looks just like a class definition except that it uses the keyword **trait**.

```
trait Equal {
      def isEqual(x: Any): Boolean
      def isNotEqual(x: Any): Boolean = !isEqual(x)
}
```

This trait consists of two methods isEqual and isNotEqual.

- No implementation for isEqual is given.
- Child classes extending the trait can give implementation for the un-implemented methods.
  - So a trait is very similar to what we have **abstract classes** in Java.

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}
```

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
   var x: Int = xc
   var y: Int = yc

   def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}
```

Scala – Traits

Example 1

```
trait Equal {
  def isEqual(x: Any): Boolean
  def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
  var x: Int = xc
  var y: Int = yc

  def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}
```

RMIT Classification: Trusted

```
trait Equal {
  def isEqual(x: Any): Boolean
  def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
  var x: Int = xc
  var y: Int = yc

  def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}
```

- obj.isInstanceOf[B] returns boolean indicating whether obj has type B or not.
- obj.asInstanceOf[B] performs the actual cast: it takes an object of type obj and returns (if possible) object of type B

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
   var x: Int = xc
   var y: Int = yc

   def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}

object Demo {
   def main(args: Array[String]) {
      val p1 = new Point(2, 3)
      val p2 = new Point(2, 4)
      val p3 = new Point(3, 3)

      println(p1.isNotEqual(p2))
      println(p1.isNotEqual(p3))
      println(p1.isNotEqual(2))
   }
}
```

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
   var x: Int = xc
   var y: Int = yc

   def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}

object Demo {
   def main(args: Array[String]) {
      val p1 = new Point(2, 3)
      val p2 = new Point(2, 4)
      val p3 = new Point(3, 3)

      println(p1.isNotEqual(p2))
      println(p1.isNotEqual(p3))
      println(p1.isNotEqual(2))
   }
}
```

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
   var x: Int = xc
   var y: Int = yc

   def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}

object Demo {
   def main(args: Array[String]) {
      val p1 = new Point(2, 3)
      val p2 = new Point(2, 4)
      val p3 = new Point(3, 3)

      println(p1.isNotEqual(p2))
      println(p1.isNotEqual(p3))
      println(p1.isNotEqual(2))
   }
}
```

2 == 3 ?
False

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
   var x: Int = xc
   var y: Int = yc

   def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}

object Demo {
   def main(args: Array[String]) {
      val p1 = new Point(2, 3)
      val p2 = new Point(2, 4)
      val p3 = new Point(3, 3)

      println(p1.isNotEqual(p2))
      println(p1.isNotEqual(p3))
      println(p1.isNotEqual(2))
   }
}
```

True

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
   var x: Int = xc
   var y: Int = yc

   def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}

object Demo {
   def main(args: Array[String]) {
      val p1 = new Point(2, 3)
      val p2 = new Point(2, 4)
      val p3 = new Point(3, 3)

      println(p1.isNotEqual(p2))     True
      println(p1.isNotEqual(p3))     False
      println(p1.isNotEqual(2))
   }
}
```

Scala – Traits

Example 1

```
trait Equal {
   def isEqual(x: Any): Boolean
   def isNotEqual(x: Any): Boolean = !isEqual(x)
}

class Point(xc: Int, yc: Int) extends Equal {
   var x: Int = xc
   var y: Int = yc

   def isEqual(obj: Any) = obj.isInstanceOf[Point] && obj.asInstanceOf[Point].x == y
}

object Demo {
   def main(args: Array[String]) {
      val p1 = new Point(2, 3)
      val p2 = new Point(2, 4)
      val p3 = new Point(3, 3)

      println(p1.isNotEqual(p2))
      println(p1.isNotEqual(p3))
      println(p1.isNotEqual(2))
   }
}
```

True
False
True

Scala – Traits

Example 2

```
import scala.collection.mutable.ArrayBuffer

trait Pet {
    val name: String
}

class Cat(val name: String) extends Pet
class Dog(val name: String) extends Pet

object Demo {
     def main(args: Array[String]) {
        val dog = new Dog("Harry")
        val cat = new Cat("Sally")
        val animals = ArrayBuffer.empty[Pet]

        animals.append(dog)
        animals.append(cat)
        animals.foreach(pet => println(pet.name))
      }
}
```

Scala – Traits

Example 2

```
import scala.collection.mutable.ArrayBuffer

trait Pet {
    val name: String
}

class Cat(val name: String) extends Pet
class Dog(val name: String) extends Pet

object Demo {
     def main(args: Array[String]) {
        val dog = new Dog("Harry")
        val cat = new Cat("Sally")
        val animals = ArrayBuffer.empty[Pet]

        animals.append(dog)
        animals.append(cat)
        animals.foreach(pet => println(pet.name))
      }
}
```

The trait Pet has an abstract field name that gets implemented by Cat and Dog in their constructors. On the last line, we call pet.name, which must be implemented in any subtype of the trait Pet.

Scala – Traits

Example 2

```
import scala.collection.mutable.ArrayBuffer

trait Pet {
    val name: String
}

class Cat(val name: String) extends Pet
class Dog(val name: String) extends Pet

object Demo {
     def main(args: Array[String]) {
        val dog = new Dog("Harry")
        val cat = new Cat("Sally")
        val animals = ArrayBuffer.empty[Pet]

        animals.append(dog)
        animals.append(cat)
        animals.foreach(pet => println(pet.name))
      }
}
```

The trait Pet has an abstract field name that gets implemented by Cat and Dog in their constructors. On the last line, we call pet.name, which must be implemented in any subtype of the trait Pet.

Scala – Traits

Example 2

```
import scala.collection.mutable.ArrayBuffer

trait Pet {
    val name: String
}

class Cat(val name: String) extends Pet
class Dog(val name: String) extends Pet

object Demo {
     def main(args: Array[String]) {
         val dog = new Dog("Harry")
         val cat = new Cat("Sally")
         val animals = ArrayBuffer.empty[Pet]

         animals.append(dog)
         animals.append(cat)
         animals.foreach(pet => println(pet.name))
      }
}
```

The trait Pet has an abstract field name that gets implemented by Cat and Dog in their constructors. On the last line, we call pet.name, which must be implemented in any subtype of the trait Pet.

Scala – Traits

Example 2

```
import scala.collection.mutable.ArrayBuffer

trait Pet {
    val name: String
}

class Cat(val name: String) extends Pet
class Dog(val name: String) extends Pet

object Demo {
     def main(args: Array[String]) {
        val dog = new Dog("Harry")
        val cat = new Cat("Sally")
        val animals = ArrayBuffer.empty[Pet]

        animals.append(dog)
        animals.append(cat)
        animals.foreach(pet => println(pet.name))
      }
}
```

The trait Pet has an abstract field name that gets implemented by Cat and Dog in their constructors. On the last line, we call pet.name, which must be implemented in any subtype of the trait Pet.

```
scala> :load /home/hadoop/Demo4.scala
Loading /home/hadoop/Demo4.scala...
import scala.collection.mutable.ArrayBuffer
defined trait Pet
defined class Cat
defined class Dog
defined object Demo4

scala> Demo4.main(Array())
Harry
Sally
```

RMIT Classification: Trusted

Scala – Traits

Example 3

```
trait Iterator[A] {
    def hasNext: Boolean
    def next(): A
}

class IntIterator(to: Int) extends Iterator[Int] {
    private var current = 0
    override def hasNext: Boolean = current < to
    override def next(): Int = {
        if (hasNext) {
            val t = current
            current += 1
            t
        } else 0
    }
}

object Demo {
    def main(args: Array[String]) {
        val iterator = new IntIterator(10)
        println(iterator.next()) // returns 0
        println(iterator.next()) // returns 1
    }
}
```

RMIT Classification: Trusted

Scala – Traits

Example 3

```
trait Iterator[A] {
    def hasNext: Boolean
    def next(): A
}

class IntIterator(to: Int) extends Iterator[Int] {
    private var current = 0
    override def hasNext: Boolean = current < to
    override def next(): Int = {
        if (hasNext) {
            val t = current
            current += 1
            t
        } else 0
    }
}

object Demo {
    def main(args: Array[String]) {
        val iterator = new IntIterator(10)
        println(iterator.next()) // returns 0
        println(iterator.next()) // returns 1
    }
}
```

- In the case you are implementing an abstract method, it is not strictly necessary to add the override modifier.
- However, in case you want to override a concrete method from the superclass, the override modifier is necessary.

RMIT Classification: Trusted

Scala – Traits

Example 3

```
trait Iterator[A] {
     def hasNext: Boolean
     def next(): A
}

class IntIterator(to: Int) extends Iterator[Int] {
     private var current = 0
     override def hasNext: Boolean = current < to
     override def next(): Int = {
          if (hasNext) {
              val t = current
              current += 1
              t
          } else 0
     }
}

object Demo {
     def main(args: Array[String]) {
          val iterator = new IntIterator(10)
          println(iterator.next()) // returns 0
          println(iterator.next()) // returns 1
     }
}
```

```
scala> :load /home/hadoop/Demo5.scala
Loading /home/hadoop/Demo5.scala...
defined trait Iterator
defined class IntIterator
defined object Demo5

scala> Demo5.main(Array())
0
1
```

RMIT Classification: Trusted

A class can mix in any number of traits

```
trait HasLegs:
  def numLegs: Int
  def walk(): Unit
  def stop() = println("Stopped walking")

trait HasTail:
  def tailColor: String
  def wagTail() = println("Tail is wagging")
  def stopTail() = println("Tail is stopped")

class IrishSetter(name: String) extends HasLegs, HasTail:
  val numLegs = 4
  val tailColor = "Red"
  def walk() = println("I'm walking")
  override def toString = s"$name is a Dog"
```

Scala - Files I/O

- Writing to a file

java.io.File is one of the objects which can be used in Scala programming to read and write files.

Scala - Files I/O

- Writing to a file

java.io.File is one of the objects which can be used in Scala programming to read and write files.

```
import java.io._
object Demo {
    def main(args: Array[String]) {
          val writer = new PrintWriter(new File("test.txt" ))
          writer.write("Hello Scala")
          writer.close()
    }
}
```

Demo3.scala

Scala - Files I/O

- Reading a Line from Command Line

```
import java.io._
object Demo {
        def main(args: Array[String]) {
                print("Please enter your input : " )
                val line = scala.io.StdIn.readLine()
                println("Thanks, you just typed: " + line)
        }
}
```

Demo6.scala

Scala - Files I/O

- Reading from a file

One can use Scala's **Source** class and its companion object to read files.

```
import scala.io.Source

object Demo {
      def main(args: Array[String]) {
            print("Please enter your input : \n" )
            val line = scala.io.StdIn.readLine()
            println("Following is the content read:\n" )
            Source.fromFile(line).foreach {
                  print
            }
       }
}
```

```
scala> :load /home/hadoop/Demo7.scala
Loading /home/hadoop/Demo7.scala...
import scala.io.Source
defined object Demo7

scala> Demo7.main(Array())
Please enter your input :
Following is the content read:

Hello Scala
```

Scala - Classes & Objects

- Basic Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " + y);
   }
}

object Demo {
   def main(args: Array[String]) {
      val pt = new Point(10, 20);

      // Move to a new location
      pt.move(10, 10);
   }
}
```

Scala - Classes & Objects

- Basic Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " +
   }
}
```

```
object Demo {
   def main(args: Array[String]) {
      val pt = new Point(10, 20);

      // Move to a new location
      pt.move(10, 10);
   }
}
```

An object is an instance of a class which is guaranteed to be unique.

- For every object in the code, an anonymous class is created, which inherits from whatever classes you declared object to implement.
- This class cannot be seen from Scala source code -- though you can get at it through reflection.

Scala - Classes & Objects

- Basic Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " + y);
   }
}

object Demo {
   def main(args: Array[String]) {
      val pt = new Point(10, 20);

      // Move to a new location
      pt.move(10, 10);
   }
}
```

RMIT Classification: Trusted

Scala - Classes & Objects

- Extending a Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " + y);
   }
}

class Location(override val xc: Int, override val yc: Int,
   val zc :Int) extends Point(xc, yc){
   var z: Int = zc

   def move(dx: Int, dy: Int, dz: Int) {
      x = x + dx
      y = y + dy
      z = z + dz
      println ("Point x location : " + x);
      println ("Point y location : " + y);
      println ("Point z location : " + z);
   }
}
```

```
object Demo {
   def main(args: Array[String]) {
      val loc = new Location(10, 20, 15);

      // Move to a new location
      loc.move(10, 10, 5);
   }
}
```

Demo8.scala

Scala - Classes & Objects

Extending a Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " + y);
   }
}
```

Location is called **subclass** of Point.

```
class Location(override val xc: Int, override val yc: Int,
   val zc :Int) extends Point(xc, yc){
   var z: Int = zc

   def move(dx: Int, dy: Int, dz: Int) {
      x = x + dx
      y = y + dy
      z = z + dz
      println ("Point x location : " + x);
      println ("Point y location : " + y);
      println ("Point z location : " + z);
   }
}
```

Point is called **superclass** of Location

```
object Demo {
   def main(args: Array[String]) {
      val loc = new Location(10, 20, 15);

      // Move to a new location
      loc.move(10, 10, 5);
   }
}
```

'**extends**' clause has two effects:

- it makes Location class inherit all non-private members from Point class, and
- it makes the type *Location* a subtype of the type *Point* class.

RMIT Classification: Trusted

Extending a Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " + y);
   }
}
```

```
class Location(override val xc: Int, override val yc: Int,
   val zc :Int) extends Point(xc, yc){
   var z: Int = zc

   def move(dx: Int, dy: Int, dz: Int) {
      x = x + dx
      y = y + dy
      z = z + dz
      println ("Point x location : " + x);
      println ("Point y location : " + y);
      println ("Point z location : " + z);
   }
}
```

```
object Demo {
   def main(args: Array[String]) {
      val loc = new Location(10, 20, 15);

      // Move to a new location
      loc.move(10, 10, 5);
   }
}
```

'**extends**' clause has two effects:

- it makes Location class inherit all non-private members from Point class, and
- it makes the type *Location* a subtype of the type *Point* class.

Scala allows the inheritance from just one class only.

RMIT Classification: Trusted

Extending a Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " + y);
   }
}
```

```
class Location(override val xc: Int, override val yc: Int,
   val zc :Int) extends Point(xc, yc){
   var z: Int = zc

   def move(dx: Int, dy: Int, dz: Int) {
      x = x + dx
      y = y + dy
      z = z + dz
      println ("Point x location : " + x);
      println ("Point y location : " + y);
      println ("Point z location : " + z);
   }
}
```

```
object Demo {
   def main(args: Array[String]) {
      val loc = new Location(10, 20, 15);

      // Move to a new location
      loc.move(10, 10, 5);
   }
}
```

'**extends**' clause has two effects:

- it makes Location class inherit all non-private members from Point class, and
- it makes the type *Location* a subtype of the type *Point* class.

Scala allows the inheritance from just one class only.

Scala - Traits

- A trait encapsulates method and field definitions, which can then be reused by mixing them into classes.
- Unlike class inheritance, in which each class must inherit from just one superclass, a class can mix in any number of traits.

Scala - Traits

- A trait encapsulates method and field definitions, which can then be reused by mixing them into classes.
- Unlike class inheritance, in which each class must inherit from just one superclass, a class can mix in any number of traits.

```
trait HasLegs:
  def numLegs: Int
  def walk(): Unit
  def stop() = println("Stopped walking")
```

```
trait HasTail:
  def tailColor: String
  def wagTail() = println("Tail is wagging")
  def stopTail() = println("Tail is stopped")
```

```
class IrishSetter(name: String) extends HasLegs, HasTail:
  val numLegs = 4
  val tailColor = "Red"
  def walk() = println("I'm walking")
  override def toString = s"$name is a Dog"
```

RMIT Classification: Trusted

Scala - Classes & Objects

Extending a Class

```
import java.io._

class Point(val xc: Int, val yc: Int) {
   var x: Int = xc
   var y: Int = yc

   def move(dx: Int, dy: Int) {
      x = x + dx
      y = y + dy
      println ("Point x location : " + x);
      println ("Point y location : " + y);
   }
}

class Location(override val xc: Int, override val yc: Int,
   val zc :Int) extends Point(xc, yc){
   var z: Int = zc

   def move(dx: Int, dy: Int, dz: Int) {
      x = x + dx
      y = y + dy
      z = z + dz
      println ("Point x location : " + x);
      println ("Point y location : " + y);
      println ("Point z location : " + z);
   }
}
```

```
object Demo {
   def main(args: Array[String]) {
      val loc = new Location(10, 20, 15);

      // Move to a new location
      loc.move(10, 10, 5);
   }
}
```

'**extends**' clause has two effects:

- it makes Location class inherit all non-private members from Point class, and
- it makes the type *Location* a subtype of the type *Point* class.

Scala allows the inheritance from just one class only.

```
scala> :load /home/hadoop/Demo8.scala
Loading /home/hadoop/Demo8.scala...
import java.io._
defined class Point
defined class Location
defined object Demo8

scala> Demo8.main(Array())
Point x location : 20
Point y location : 30
Point z location : 20
```

Scala - Access Modifiers

- Private Members

A private member is visible only inside the class or object that contains the member definition.

```
class Outer {
   class Inner {
     private def f() { println("f") }

     class InnerMost {
        f() // OK
     }
   }
   (new Inner).f() // Error: f is not accessible
}
```



Scala - Access Modifiers

- Private Members

A private member is visible only inside the class or object that contains the member definition.

```
class Outer {
   class Inner {
     private def f() { println("f") }

     class InnerMost {
        f() // OK
     }
   }
   (new Inner).f()  or: f is not accessible
}
```





Scala - Access Modifiers

- Protected Members

A protected member is only accessible from subclasses of the class in which the member is defined.

```
package p {
   class Super {
      protected def f() { println("f") }
   }

   class Sub extends Super {
      f()
   }

   class Other {
      (new Super).f() // Error: f is not accessible
   }
}
```



Scala - Access Modifiers

- Protected Members

A protected member is only accessible from subclasses of the class in which the member is defined.

```
package p {
   class Super {
      protected def f() { println("f") }
   }

   class Sub extends Super {
      f()
   }

   class Other {
      (new Super).f() // f is not accessible
   }
}
```





Scala - Access Modifiers

- Protected Members

A protected member is only accessible from subclasses of the class in which the member is defined.

Package in Scala is a mechanism to encapsulate a group of classes, sub packages, traits and objects. It basically provides namespace to put our code in a different files and directories.

```
package p {
   class Super {
      protected def f() { println("f") }
   }

   class Sub extends Super {
      f()
   }

   class Other {
      (new Super).f() // f is not accessible
   }
}
```





Scala - Access Modifiers

import clauses are for accessing members (classes, traits, functions, etc.) in other packages.

- Protected Members

A protected member is only accessible from subclasses of the class in which the member is defined.

Package in Scala is a mechanism to encapsulate a group of classes, sub packages, traits and objects. It basically provides namespace to put our code in a different files and directories.

```
package p {
   class Super {
      protected def f() { println("f") }
   }

   class Sub extends Super {
      f()
   }

   class Other {
      (new Super).f() // f is not accessible
   }
}
```





Scala - Access Modifiers

- Public Members

Unlike private and protected members, it is not required to specify Public keyword for Public members. There is no explicit modifier for public members. Such members can be accessed from anywhere.

```
class Outer {
   class Inner {
      def f() { println("f") }

      class InnerMost {
         f() // OK
      }
   }
   (new Inner).f() // OK because now f() is public
}
```

Scala - Access Modifiers

- Public Members

Unlike private and protected members, it is not required to specify Public keyword for Public members. There is no explicit modifier for public members. Such members can be accessed from anywhere.

```
class Outer {
   class Inner {
      def f() { println("f") }

      class InnerMost {
         f() // OK
      }
   }
   (new Inner).f() // OK because now f() is public
}
```

Arithmetic Operators

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiplies both operands	A * B will give 200
/	Divides numerator by de-numerator	B / A will give 2
%	Modulus operator finds the remainder after division of one number by another	B % A will give 0

Relational Operators

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Logical Operators

Operator	Description	Example
&&	It is called Logical AND operator. If both the operands are non zero then condition becomes true.	(A && B) is false.
||	It is called Logical OR Operator. If any of the two operands is non zero then condition becomes true.	(A || B) is true.
!	It is called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true.

Bitwise Operators

p	q	p & q	p | q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

RMIT Classification: Trusted

Bitwise Operators

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12, which is 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) will give 61, which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49, which is 0011 0001
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61, which is 1100 0011 in 2's complement form due to a signed binary number.
<<	Binary Left Shift Operator. The bit positions of the left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240, which is 1111 0000
>>	Binary Right Shift Operator. The Bit positions of the left operand value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15, which is 1111
>>>	Shift right zero fill operator. The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.	A >>>2 will give 15 which is 0000 1111

Assignment Operators

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand	C = A + B will assign value of A + B into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator	C &= 2 is same as C = C & 2
^=	bitwise exclusive OR and assignment operator	C ^= 2 is same as C = C ^ 2
|=	bitwise inclusive OR and assignment operator	C |= 2 is same as C = C | 2

Scala - Interpolator

String Interpolation allows users to embed variable references directly in *processed* string literals.

Scala - Interpolator

String Interpolation allows users to embed variable references directly in *processed* string literals.

- The 's' String Interpolator

's' interpolator in appending String variable ($name) to a normal String (Hello) in println statement.

```
val name = "James"
```

Scala - Interpolator

String Interpolation allows users to embed variable references directly in *processed* string literals.

- The 's' String Interpolator

's' interpolator in appending String variable ($name) to a normal String (Hello) in println statement.

```
val name = "James"
println(s"Hello, $name") //output: Hello, James
```

```
println(s"1 + 1 = ${1 + 1}")
```

```
println(s"New offers starting at $$14.99")
```

Scala - Interpolator

String Interpolation allows users to embed variable references directly in *processed* string literals.

- The 's' String Interpolator

's' interpolator in appending String variable ($name) to a normal String (Hello) in println statement.

```
val name = "James"
println(s"Hello, $name") //output: Hello, James
```

```
println(s"1 + 1 = ${1 + 1}")
```

```
println(s"New offers starting at $$14.99")
```

```
scala> println(s"Hello, $name")
Hello, James

scala> println(s"1 + 1 = ${1+1}")
1 + 1 = 2

scala> println(s"New offers starting at $$14.99")
New offers starting at $14.99
```

Scala - Interpolator

String Interpolation allows users to embed variable references directly in *processed* string literals.

- The ‘ f ’ Interpolator

  Using ‘f’ interpolator, all variable references should be followed by the **printf** style format specifiers such as %d, %i, %f, etc.

```
val height = 1.9d  // data type double
val name = "James"
println(f"$name%s is $height%2.2f meters tall") //James is 1.90 meters tall
```

Scala - Interpolator

String Interpolation allows users to embed variable references directly in *processed* string literals.

- The ‘ f ’ Interpolator

  Using ‘f’ interpolator, all variable references should be followed by the **printf** style format specifiers such as %d, %i, %f, etc.

```
val height = 1.9d  // data type double
val name = "James"
println(f"$name%s is $height%2.2f meters tall") //James is 1.90 meters tall
```

The format string "%4.2f" says: format with 2 digits after radix point and use at least 4 characters for the full string.

```
 println(f"$name%s is ---$height%4.2f--- meters tall")
James is ---1.90--- meters tall

 println(f"$name%s is ---$height%10.2f--- meters tall")
James is ---      1.90--- meters tall
```

Scala - Interpolator

String Interpolation allows users to embed variable references directly in *processed* string literals.

- The ‘ f ’ Interpolator

  Using ‘f’ interpolator, all variable references should be followed by the **printf** style format specifiers such as %d, %i, %f, etc.

```
val height = 1.9d  // data type double
val name = "James"
println(f"$name%s is $height%2.2f meters tall") //James is 1.90 meters tall
```

```
scala> val height = 1.9d
height: Double = 1.9

scala> val name = "James"
name: String = James

scala> println(f"$name%s is $height%2.2f meters tall")
James is 1.90 meters tall
```

Execute Scala Programs

- Two modes
  - Interactive mode
  - Script mode

Type the following text to the right of the Scala prompt and press the Enter key

```
scala> println("Hello, Scala!");
```

Interactive mode in AWS EMR

- $spark-shell

```
[hadoop@ip-192-168-18-110 ~]$ spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
20/08/08 06:41:05 WARN Client: Neither spark.yarn.jars nor spark.yarn.archive is set, falling ba
ck to uploading libraries under SPARK_HOME.
Spark context Web UI available at http://ip-192-168-18-110.ec2.internal:4040
Spark context available as 'sc' (master = yarn, app id = application_1596865122117_0002).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 2.4.5-amzn-0
      /_/

Using Scala version 2.11.12 (OpenJDK 64-Bit Server VM, Java 1.8.0_252)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

```
scala> println("Hello, Scala!")
Hello, Scala!

scala>
```

Execute Scala Programs

- Two modes
  - interactive mode
  - script mode

Use the following instructions to write a Scala program in script mode. Open notepad and add the following code into it.

```
object HelloWorld {
     def main(args: Array[String]) {
          println("Hello, world!")
          // prints Hello World
     }
}
```

Save the file as − HelloWorld.scala.

```
\> scalac HelloWorld.scala
\> scala HelloWorld
```

```
Hello, World!
```

Script mode in AWS EMR

AWS EMR, with Spark installed, does not yet have a Scala compiler

Script mode in AWS EMR

AWS EMR, with Spark installed, does not yet have a Scala compiler

- Method 1: load and run

```
scala> :load /home/hadoop/Demo.scala
Loading /home/hadoop/Demo.scala...
import java.io._
defined class Point
defined class Location
defined object Demo

scala> Demo.main(Array())
Point x location : 20
Point y location : 30
Point z location : 20

scala>
```

Script mode in AWS EMR

AWS EMR, with Spark installed, does not yet have a Scala compiler

- Method 1: load and run

```
scala> :load /home/hadoop/Demo.scala
Loading /home/hadoop/Demo.scala...
import java.io._
defined class Point
defined class Location
defined object Demo

scala> Demo.main(Array())
Point x location : 20
Point y location : 30
Point z location : 20

scala>
```

Script mode in AWS EMR

AWS EMR, with Spark installed, does not yet have a Scala compiler

- Method 1: load and run

```
scala> :load /home/hadoop/Demo.scala
Loading /home/hadoop/Demo.scala...
import java.io._
defined class Point
defined class Location
defined object Demo

scala> Demo.main(Array())
Point x location : 20
Point y location : 30
Point z location : 20

scala>
```

- Method 2: compile Demo.scala into a JAR file and use "spark-submit" command to run it.

End of Part II

Part III
Scala Exercises

“in-class Scala exercises.pdf”